

# Departamento Electrónica — Técnicas Digitales II

---

## Actividad de Formación Práctica N°3 — LAB

**Tema:** Creación de drivers GPIO en STM32CubeIDE.

**Profesor:** Ing. Rubén Darío Mansilla **ATTP:** Ing. Lucas Abdala

**AFP 3 — Grupo 3 — TDII 2026**

Integrantes:

- Plaate Iván
- *(completar compañeros)*
- *(completar compañeros)*

---

## 1. Introducción

El presente informe documenta el desarrollo de la AFP N°3 de laboratorio, cuyo objetivo fue implementar un **driver para el módulo GPIO** de la placa de desarrollo **STM32 NUCLEO-F429ZI**, siguiendo la guía de la cátedra ("Creación de driver GPIO"), e integrarlo en las aplicaciones de encendido de LEDs desarrolladas previamente, reemplazando las llamadas directas a la HAL por funciones del driver.

El criterio de diseño aplicado fue la **portabilidad**: el driver se implementó de forma que pueda reutilizarse en cualquier puerto, pin y placa sin modificar su código interno, y que pueda evolucionar y cargarse en las prácticas siguientes ya pulido.

## 2. Desarrollo del driver API\_GPIO

---

El driver se creó en la estructura de la cátedra:

```
Drivers/  
└─ API/  
    └─ Inc/  
        └─ API_GPIO.h  
    └─ Src/  
        └─ API_GPIO.c
```

### 2.1. Tipos de datos exportados

La decisión de diseño central fue identificar cada pin físico mediante una estructura que transporta **puerto y pin** juntos:

```
typedef struct
{
    GPIO_TypeDef * port;    /* Puerto del pin (ej: LD1_GPIO_Port) */
    uint16_t pin;          /* Mascara del pin (ej: LD1_Pin) */
} gpioPin_t;

typedef gpioPin_t led_t;    /* Un led es un pin de salida */
typedef gpioPin_t button_t; /* Un pulsador es un pin de entrada */
typedef bool buttonStatus_t; /* Estado del pulsador */
```

A diferencia del enfoque en el que `led_t` es solamente la máscara del pin (`uint16_t`) — que obliga a asumir un puerto fijo (típicamente `GPIOB`) dentro del driver —, la estructura `gpioPin_t` permite que las funciones reciban el pin completo por parámetro. De esta forma el driver **no queda atado a ningún puerto ni placa en particular** y es realmente reutilizable.

Los valores concretos de puerto y pin se toman siempre de los nombres generados por CubeMX (`LD1_GPIO_Port`, `LD1_Pin`, `USER_Btn_GPIO_Port`, `USER_Btn_Pin`), sin literales hardcodeados.

## 2.2. Funciones exportadas

| Función                                       | Descripción                                                                                                   |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>MX_GPIO_Init()</code>                   | Inicialización de los pines GPIO (reubicada desde <code>main.c</code> al driver, según la guía de la cátedra) |
| <code>writeLedOn_GPIO(led_t led)</code>       | Enciende el led indicado                                                                                      |
| <code>writeLedOff_GPIO(led_t led)</code>      | Apaga el led indicado                                                                                         |
| <code>toggleLed_GPIO(led_t led)</code>        | Invierte el estado del led indicado                                                                           |
| <code>readButton_GPIO(button_t button)</code> | Devuelve el estado del pulsador ( <code>true</code> = presionado)                                             |

Cada función encapsula su primitiva HAL correspondiente (`HAL_GPIO_WritePin`, `HAL_GPIO_TogglePin`, `HAL_GPIO_ReadPin`), de modo que la aplicación **no contiene ninguna llamada directa a la HAL para el manejo del módulo GPIO**.

## 3. Aplicaciones modificadas

Se integró el driver en las cuatro aplicaciones de secuencias de LEDs de la AFP1, respetando el formato de nombre `App_3_Y_Grupo_3_2026`. En todas ellas la aplicación es de carácter general, mediante un vector de `led_t` que permite extender la cantidad de LEDs con mínimas modificaciones.

### 3.1. App\_3\_1 — Secuencia básica

Enciende y apaga secuencialmente los tres LEDs onboard (LD1 verde → LD2 azul → LD3 rojo), 200 ms encendido y 200 ms apagado por LED, en forma circular. Es la refactorización directa de la App 1.1 reemplazando las llamadas HAL por funciones del driver.

### 3.2. App\_3\_2 — Inversión de sentido

Misma secuencia que App\_3\_1, pero el pulsador de usuario invierte el sentido de recorrido cada vez que se presiona. La lectura del pulsador se realiza en pasos de 10 ms dentro de las esperas, de modo que la respuesta al botón es inmediata.

### 3.3. App\_3\_3 — Cuatro secuencias alternadas

El pulsador alterna circularmente entre cuatro secuencias:

- **Secuencia 1:** encendido secuencial con alternancia de 150 ms.
- **Secuencia 2:** los tres LEDs parpadean simultáneamente con alternancia de 300 ms.
- **Secuencia 3:** parpadeo con períodos independientes — LD1 a 100 ms, LD2 a 300 ms y LD3 a 600 ms, implementado con un tick base de 100 ms.
- **Secuencia 4:** LD1 y LD3 parpadean juntos mientras LD2 lo hace en forma inversa, con alternancia de 150 ms.

Al cambiar de secuencia se apagan todos los LEDs y se reinicia el estado para que la nueva arranque desde una condición conocida.

### 3.4. App\_3\_4 — Frecuencia de parpadeo variable

Los tres LEDs parpadean simultáneamente y el pulsador cambia el tiempo de alternancia de forma secuencial entre cuatro valores predefinidos: 100 ms → 250 ms → 500 ms → 1000 ms → vuelta a 100 ms. Los tiempos se almacenan en un vector, por lo que la aplicación es extensible a otros valores con mínimas modificaciones.

## 4. Observaciones

---

- La inicialización `MX_GPIO_Init()` se trasladó al driver (`API_GPIO.c`), por lo que `main.c` quedó libre de configuración y acceso directo al hardware.
- La detección del pulsador se implementa por flanco ascendente, leyéndolo en pasos de 10 ms durante las esperas: la respuesta es inmediata sin bloquear más que el paso de lectura.

- El rebote mecánico del pulsador no se trata en esta práctica; el antirrebote por máquina de estados se incorpora en la AFP5. En estas aplicaciones una doble lectura por rebote no altera el comportamiento perceptible.
- El driver quedó listo para reutilizarse en las prácticas siguientes (AFP4 y AFP5) sin modificaciones, dado que no depende de puertos, pines ni tiempos concretos.

## 5. Repositorio

---

Repositorio grupal en GitHub:

[https://github.com/ivanplaate-oss/Grupo\\_3\\_TDII\\_2026](https://github.com/ivanplaate-oss/Grupo_3_TDII_2026)

Las aplicaciones se encuentran en la carpeta `AFP_3_TDII_2026/` :

- `AFP_3_TDII_2026/App_3_1_Grupo_3_2026` — refactor de App 1.1
- `AFP_3_TDII_2026/App_3_2_Grupo_3_2026` — refactor de App 1.2
- `AFP_3_TDII_2026/App_3_3_Grupo_3_2026` — refactor de App 1.3
- `AFP_3_TDII_2026/App_3_4_Grupo_3_2026` — refactor de App 1.4

## 6. Conclusiones

---

Se implementó un driver GPIO portable y se lo integró en las cuatro aplicaciones, verificando el funcionamiento correcto en la placa NUCLEO-F429ZI. La separación entre la capa de aplicación y el hardware a través del driver produce código más limpio, legible y reutilizable, cumpliendo la filosofía de programación profesional sin hardcodear que plantea la materia.